Session 6

1.

Given a MongoDB collection 'orders' with documents containing fields: userId, items (array of {product, quantity, price}), and orderDate, use the $match stage to find all orders placed in the last 30 days.

```
db.orders.aggregate([
 {
   $match: {
     orderDate: {
       $gte: new Date(
         new Date().setDate(new Date().getDate() - 30)
       )
     }
   }
 }
]);
```

2.

Using the 'orders' collection, write an aggregation pipeline that groups orders by userId and calculates the total amount spent by each user using the $group stage.

Hint: Use $sum to total the value of all items in each user's orders.

```
> db.orders.aggregate([
    {
      $unwind: "$items"
    },
    {
      $group: {
        _id: "$userId",
        totalSpent: {
          $sum: {
            $multiply: ["$items.quantity", "$items.price"]
          }
        }
      }
    }
  ]);
< {
    _id: 101,
    totalSpent: 53000
```

```
{
  _id: 101,
  totalSpent: 53000
}
{
  _id: 103,
  totalSpent: 6000
}
{
  _id: 102,
  totalSpent: 25000
}
usinessDB>
```

3.

Create an aggregation pipeline on a 'restaurants' collection (fields: name, cuisine, city, rating) to find the top 3 highest-rated cafes in Ahmedabad. Use $match for cuisine, $sort for rating, and $limit for top results.

```
{
  _id: ObjectId('6a3cd055572d7c8617aa9ccf'),
  name: 'Urban Cafe',
  cuisine: 'Cafe',
  city: 'Ahmedabad',
  rating: 4.9
}
{
  _id: ObjectId('6a3cd055572d7c8617aa9ccd'),
  name: 'Cafe Mocha',
  cuisine: 'Cafe',
  city: 'Ahmedabad',
  rating: 4.8
}
{
  _id: ObjectId('6a3cd055572d7c8617aa9cce'),
  name: 'Coffee House',
  cuisine: 'Cafe',
  city: 'Ahmedabad',
  rating: 4.6
```

4.

In a 'movies' collection (fields: title, genre, boxOffice), use $group to calculate the total box office collection for each genre and then use $project to display only genre and totalCollection fields in the output.

Constraint: Do not include the default _id field in your final projection.

db.movies.aggregate([

{

$group: {

_id: "$genre",

totalCollection: {

$sum: "$boxOffice"

}

}

},

```
 {
   $project: {
     _id: 0,
     genre: "$_id",
     totalCollection: 1
   }
 }
]);
```

5.

Use ChatGPT or GitHub Copilot to generate an aggregation pipeline that produces a daily sales report from a 'payments' collection (fields: userId, amount, paymentDate), showing total sales per day for the last week. Paste the generated code and briefly describe any changes you made to fit your collection's structure.

```
db.payments.aggregate([
 {
   $match: {
     paymentDate: {
       $gte: new Date(
         new Date().setDate(new Date().getDate() - 7)
       )
     }
   }
 },
 {
   $group: {
     _id: {
       $dateToString: {
         format: "%Y-%m-%d",
         date: "$paymentDate"
```

```
      }
    },
    totalSales: {
      $sum: "$amount"
    }
  }
 },
 {
  $project: {
    _id: 0,
    date: "$_id",
    totalSales: 1
  }
 },
 {
  $sort: {
    date: 1
  }
 }
]);
```